

プログラミング①

2026 年度高 2 情報

2026/05/02

机に貼ってある番号に従って座ってください
PC の準備は不要です
課題を出す方は前の箱に

前回の復習

解答例は次回配布予定.

符号付き 2 進数の復習

- 最上位 bit が 0 なら正の数. そのまま 10 進数に直す
- 最上位 bit が 1 なら負の数
- 符号を外すには, 2 の補数をとる必要がある
(正の数を負の数に直すときも同じ)
2 の補数をとるとは, 各 bit の 0 と 1 を入れ替えて 1 を足す操作のこと
- 4bit 符号付き 2 進数 1110 は, 2 の補数を取ると, $0001+1=0010$ となるので, 1110 は -2 を意味していることが分かる

前回の復習

オーバーフローの定義

- コンピュータは演算できる数の範囲が決まっている. それを超えると**オーバーフロー**となり, 誤差が生じる (想定外の値が出力される)
- 想定の桁数を越えたからといって, 必ずオーバーフローになるわけではない. 扱う数の範囲を超えているかどうかが基準.

前回の復習

オーバーフローの定義

- 想定の桁数を越えたからといって、必ずオーバーフローになるわけではない。扱う数の範囲を超えているかどうかが基準。
- 例えば課題 3(4) の $1100_{(2)} + 1110_{(2)}$ などが桁あふれが起こるが、オーバーフローではない例。

$$-4_{(10)} = 1100_{(2)} \xrightarrow{2\text{の補数}} 0100_{(2)} = 4_{(10)}$$

$$-2_{(10)} = 1110_{(2)} \xrightarrow{2\text{の補数}} 0010_{(2)} = 2_{(10)}$$

一方,

$$1100_{(2)} + 1110_{(2)} = 1\boxed{1010}_{(2)} = 1010_{(2)} = -6_{(10)} \quad (\because 1010 \xrightarrow{2\text{の補数}} 0110 = 6_{(10)})$$

∴ この計算は桁あふれは起きているが、正しく計算できている
(つまり計算可能である)

プログラミングとは

アルゴリズム

- 問題解決のために決められた手段や方法のことを**アルゴリズム**という
 - 連立1次方程式を解くための加減法
 - エラトステネス (Eratosthenes) のふるい

プログラミング

- アルゴリズムをコンピュータに命令を指示する「言語」を用いて表したものを**プログラム**という
- プログラムをつくることを**プログラミング**という

プログラミングをやってみよう

留意事項

- 個人のアドレス (学校用も含む) で何かにログインしない (僕が仕様を知らないので)
- 経験者や理解が進んだ人は積極的に周囲で困っている人を助けてあげてください

プログラミングをやってみよう

実行環境へ移動

- Google Chrome を起動 (左下の窓のマークを押して, G のところにある)
 - `paiza.io/ja` と検索
 - コードを作成をクリック
 - 左上の Python と書かれているところを `C#` に変更
- できたコードを実行してみよう

コードの説明

```
1 public class Hello{  
2     public static void Main(){  
3         System.Console.WriteLine("Hello C#");  
4     }  
5 }
```

コードの中身 (おまじないのように毎度書くもの)

- **public class Hello**
 - プログラムの「外枠 (クラス)」の名前の定義
 - C#ではこれから始めるのが原則
- **public static void Main()**
 - コンピュータはこの場所を探して実行を開始する

コードの説明

```
1 public class Hello{  
2     public static void Main(){  
3         System.Console.WriteLine("Hello C#");  
4     }  
5 }
```

コードの中身 (こっちは大事！)

- **System.Console.WriteLine("...")**
 - 「画面（コンソール）に”” の中身の文字をそのまま表示せよ」
という具体的な命令
- **{ } (波カッコ) と ; (セミコロン)**
 - { } は命令が届く範囲を意味する
 - ; は命令の終わりを意味する

プログラミングがうまくいかないときには

- 全角文字がコード中に入っていないかを確認しよう
- 余計なスペースなどが入っていないかを確認しよう
- セミコロンが文末についているかを確認しよう
- 括弧が閉じていることを確認しよう
- 括弧の種類が正しいかを確認しよう

実際に書いてみよう-1

//はコメントアウトといい、その行の//のあとの内容は読まれない。
メモに便利。以下のコードの緑の部分は書かなくてもよい。

```
1 public class Add{
2     public static void Main(){
3         int a = 10;
4         // 変数aを整数型として宣言して(int a)
5         //10を入れる(=10)
6         int b = 20;
7         // 変数bを整数型として宣言して20を入れる
8         int sum = a + b;
9         // 変数sumを整数型として宣言してa+b=30を入れる
10        System.Console.WriteLine(sum); // 結果を表示
11        System.Console.WriteLine("sum"); //これはどう?
12    }
13 }
```

実際に書いてみよう-1

留意事項

- `System.Console.WriteLine(sum)`
これは `sum` の中身を表示
- `System.Console.WriteLine("sum")`
これは `sum` の文字そのものを表示

変数

- 値に名前を付けることができ、その参照を**変数**という
- 変数の名前を**変数名**という。先ほどの `a`, `b`, `sum` は変数名。
- 値に名前を付ける構文 (`a=10` など) の文を**代入文**または**変数定義**という。

実際に書いてみよう-2

緑の文字は書かなくてよい

```
1 public class Add{
2     public static void Main(){
3         int a = 10;
4         int b = 20;
5         int sum = a + b;
6         System.Console.WriteLine(sum); // 結果を表示
7         a = 30; // 既にaという変数があるので, intを付けて
            宣言しない.
8         sum = a + b;
9         System.Console.WriteLine(sum); // 結果を表示
10    }
11 }
```

上から読まれるので, 同じ文でもどこで書くかによって結果が変わる

実際に書いてみよう-3

```
1 public class Add{  
2     public static void Main(){  
3         int a = 10;  
4         int b = 20;  
5         a = a + 1; // 既にaという変数があるので, intを付け  
6         て宣言しない.  
7         int sum = a + b;  
8         System.Console.WriteLine(sum); // 結果を表示  
9     }  
}
```

$a = a + 1$ などの表記はプログラミングではありうる。
右辺の a は既知の情報 (今回は 10) をもった a で, 左辺の a はそれで再定義される. $\therefore$ 出力結果は 31.

変数の名づけのルール

変数の名前でだめなもの

- 予約語
 - int, double などのデータの型 (種類) を決める言葉
 - if や for など既に特別な制御に関係があるもの
- 演算子
 - + や -, *, / などの演算記号
- 数字から始まるもの `1st_score` などはダメ
- 全角文字や半角スペース・全角スペースを含むもの (全角文字もできるようになっているが, 避けるべきである)

本日の課題

内容

- もう一度2進数の計算 (今度は8bitで挑戦)
操作の内容は変わらない
- プログラムを読んでみる (今後はこういう課題が多くなるはず)